LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS4051NI\CC4059NI Fundamental of Computing

70% Individual Coursework

Submission: Milestone 1

Academic Semester: Spring Semester 2026

Credit: 15 credit semester long module

Student Name: Rijan Bhattarai

London Met ID: 25029641

College ID: NP01AI4A250028

Google Link:

https://drive.google.com/file/d/1SvY2PRgpmb2oDU1JgpvJIeeMAYKXgPTB/view?usp=sharing

Assignment Due Date: Friday, April 24, 2026

Assignment Submission Date: Friday, April 24, 2026

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Contents 2
Table of Figures 3
Table of Tables 4
1. Introduction 1
1.1 Project Description 1
1.2 Project Goals 1
1.2 Project Objective 1
1.3 Technologies Used 2
2. Algorithm 3
3. Flowchart 5
4. Data Structures 6
4.1 Primitive Data Types 6
4.2 Collection Data Types 6
4.3 Data Structures Used 7
5. Program 8
5.1 Medicine Info File 8
5.2 Read File Function 9
5.3 Display Function 9
5.4 Output 10
6. Conclusion 11
6.1 Challenges Faced 11
7. Bibliography 12

Table of Figures

Figure 1: Python Logo 2
Figure 2: Python IDLE 3
Figure 3: Flowchart 5
Figure 4: Medicine_info.txt File 8
Figure 5: Read File Function - Program 1 9
Figure 6: Display Function - Program 2 10
Figure 7: Output of Program 10

Table of Tables

Table 1: Primitive Data Types 6
Table 2: Collection Data Types 7

1. Introduction

1.1 Project Description

The objective of the project is to create an application which can read the contents of medicine inventory text file and display all available medicine in stock, with each transaction the file is continuously updated to ensure latest availability. With each transaction that occurs in throughout a time period an invoice is generated per customer which including the details medicine name, brand, quantity of the medicine purchased, medicine bought in strips or tablets, applicable discount depending on the type of medicine and quantity, and the total amount of the medicine purchased by the customer.

1.2 Project Goals

The primary goal of this project is to improve customer support experience and staffs' productivity by making sure that the stock availability of medicines is readily available to view, so that no extra time is wasted on searching for medicines when they are out of stock.

1.2 Project Objective

The primary objective of the project is to automate the tracking, management, and optimization of medical supplies, and pharmaceutical supplies. These applications remove the need for manual error-prone methods and increases the customer satisfactory with the business and increases productivity for employees by allowing facilities to prevent stockouts, minimize waste from expired medicines and maintain regulatory compliance with national and international standards such as HIPPA, FDA etc.

Key functions of objective include:

- Real-time monitoring of stock levels with automated low-stock alerts
- Profit and loss tracking with a report system that is seamlessly integrated into the program for ease of use

- Data driven monitoring due to report generation nature of the program the business can easily track profit, loss, clients and recent trends with the appropriate information required for the business for future.

1.3 Technologies Used

The language of choice to test the prototype and build the program is python. Python is a clear and powerful object-oriented programming language, comparable to Perl, Ruby, Scheme, or Java (Kelly, 2019). Python also being a cross-platform language having simple syntax with an extensive libraries and ecosystem whilst also having simple syntax for productivity has made it industries favourite and my favourite while developing this program. (Sayeth Sabbith, 2019)

Figure 1: Python Logo

Python Integrated Development and Learning Environment (IDLE) is a built in tool for writing, running, and debugging python programs. Python IDLE was used in order to develop the program from the ground up as it requires not extra installation and no further set up to get it up and running. The components of Python IDLE are:

- **Editor:** Designed for writing, editing, and, debugging python code.
- **Python Shell (IDLE):** For interactive python commands.

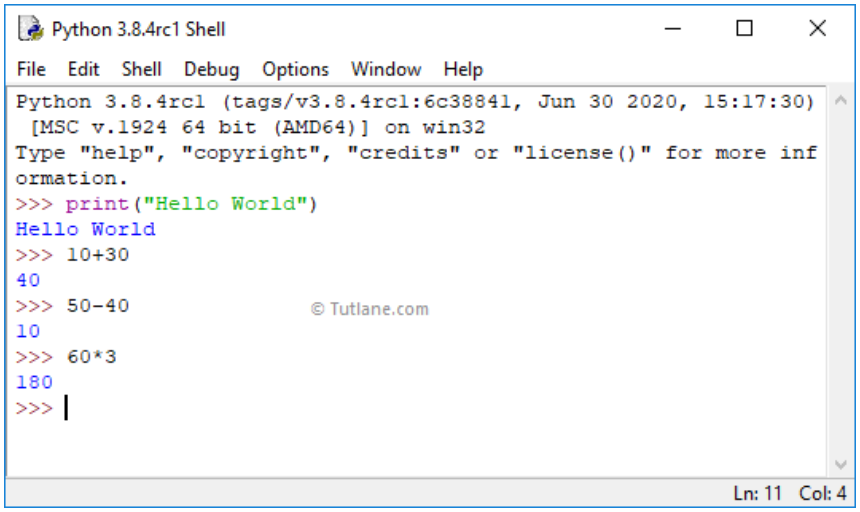


Figure 2: Python IDLE

2. Algorithm

Finite set of rules that a computer has to follow to achieve a specific task is defined as algorithm (Knuth, 1997)

Step 1

1.1 Initialize file path to “medicine_info.txt”

Step 2 – Define Function read_file

2.1.1 Try to open the file in read mode

2.1.2 If file doesn’t exist raise an error and return an empty dictionary

2.2 Read lines of the file and store it in a variable

2.3.1 Iterate over the lines using a for loop

2.3.2 Split the line into a string array

2.3.3 Parse over the array to store necessary information in appropriate variables

2.4 Store the values into a dictionary where index are keys and list of medicine information are values

2.5 Return the dictionary

Step 3 – Display Function

Step 3.1 Define the function to take read_file as an argument – call back function

Step 3.2 Store the content returned from read_file function in a variable

Step 3.3 Display the headers to show data in a tabular format

Step 3.4 Iterate over the dictionary

Step 3.4.1 Display the content of dictionary with a tabular format

Step 4 – Display contents

Step 4.1 Call display function

Step 5 – User Input

Step 5.1 In a while loop prompt the user to enter medicine information until they are satisfied

Step 5.1 User is prompted to enter yes to continue adding more medicine in their cart or entering no to break out of the loop.

Step 5.1 Store the user input into a list

Step 6 – Create Invoice Function

Step 6.1 User input is taken as argument for the function

Step 6.2 Iterate over user input to find relevant fields such as tablet or strip, number of tablets, and price to calculate cost.

Step 6.3 Apply discounts to applicable medicines to calculate discount amount

Step 6.4 Calculate total cost using cost and discount amount.

Step 7 – Generate Invoice File

Step 7.1 Take user input, invoice function as argument for the function

Step 7.2 Write into a file called invoice.txt having fields medicine name, brand name, total cost, date of purchase, and bill number.

Step 7.3 Append in the file

3. Flowchart

A flowchart is a visual representation of the flow of the program that maps out a process, system, or algorithm step-by-step using standardized symbols, shapes, and arrows (ISO, 1998). It illustrates the sequence of action, decision, and data flow from start to finish, helping to document, analyze and improve complex workflows. The components used in flowchart are shapes such as rectangles, diamonds, and ovals connected by arrow to show direction (American National Standard, 1973). The primary symbols represent:

- **Oval (Terminal)**: Represent the start of end of a process
- **Rectangle (Process)**: Represents a step or action
- **Diamond (Decision):** Indicates a decision part of the program
- **Parallelogram (I/O)**: Represents input or output of the program

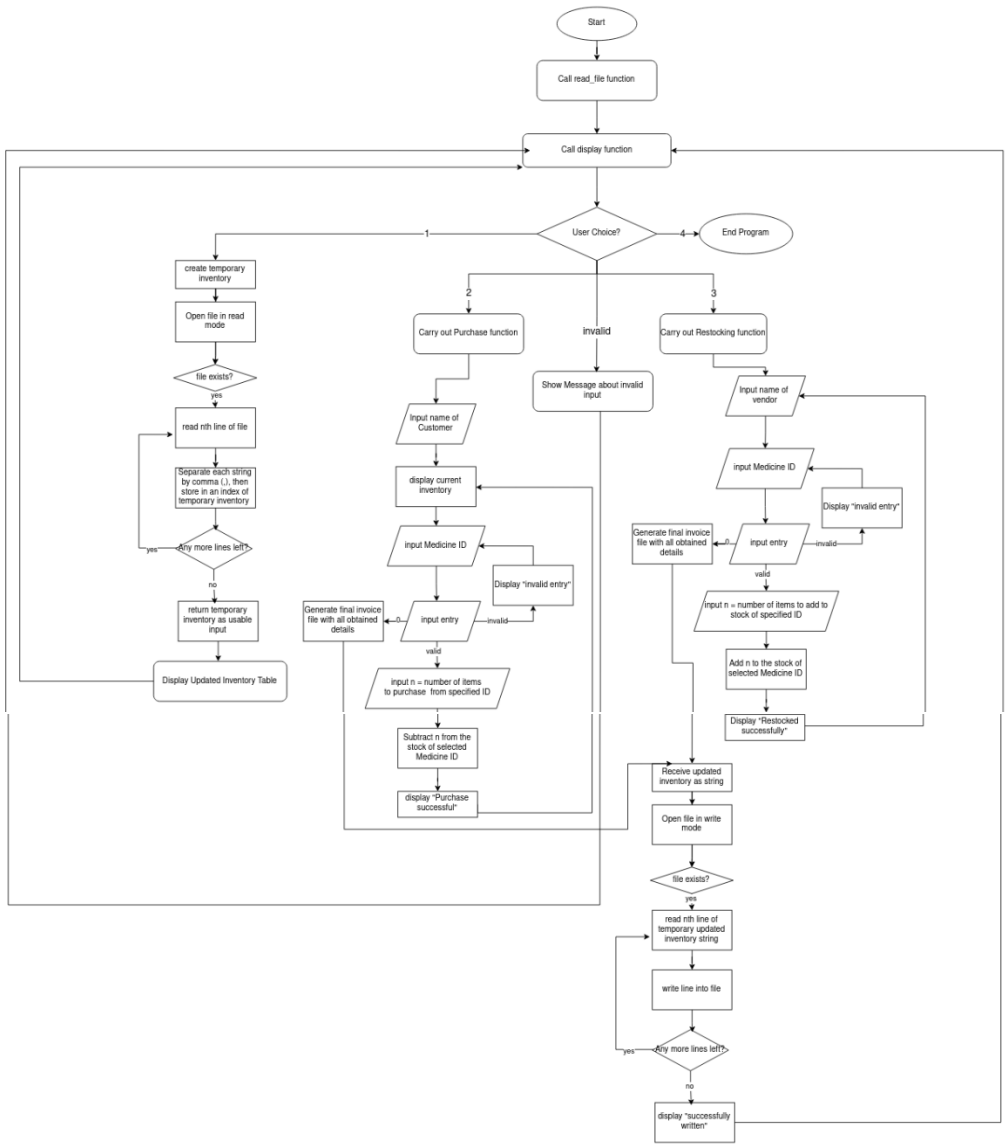


Figure 3: Flowchart

4. Data Structures

Python data types are classified into two types, they are primitive data types and collection data types. They are described in brief below:

- **Primitive Data Types** represent the basic, immutable building blocks for a program, that store a single value. Primitive data type consists of integers – *int,* floating point numbers – *float*, Boolean - *bool* and strings – *str.*
- **Collection Data Types** represent mutable blocks which can store multiple values within a single variable. They allow indexing and iteration of values stored. Collection data type consists of list, dictionary, set, tuples.

The table below lists all built-in data types which are either primitive or collection-oriented according to the official Python documentation.

4.1 Primitive Data Types

Data Type	Description	Example
Integers – int	Whole numbers of infinite length (unlimited precision)	age = 20
Floating Point Numbers – float	Double precision data type which holds decimal values	price = 20
Complex	Numbers which are paired with real and imaginary part	z = 2 + 3j
Boolean – bool	Boolean Values (True or False)	isVerified = False
Strings – str	Immutable sequence of Unicode characters	name = "Lebron"
NoneType	Represents null value (None)	val = None

Table 1: Primitive Data Types

4.2 Collection Data Types

Data Type	Description	Example
List	Mutable, ordered sequence	l = [1, 2, 3]
Tuple	Immutable, ordered sequence	tup = (1, 2)
Dictionary	Mapping data type of key-value pairs	d = {1: “Lakers”, 2: “Cavs”}
Set	Mutable, unordered collection of elements	s = {1, 2, 3, 4}
Frozenset	Immutable set	fs = frozenset((1, 2, 3))
Range	Immutable sequence of numbers - (start, stop, step)	range(4)

Bytes	Immutable sequence of bytes	x = bytes(5)
ByteArray	Mutable Sequence of bytes	y = bytearray(5)
Memory View	View of another objects binary data	memoryview(b'abc')

Table 2: Collection Data Types

4.3 Data Structures Used

This section covers the data structures used in the project to accomplish the goals and objectives of the project described in sections **1.2** and **1.3**.

Data Type	Category	Purpose in Program
Int	Primitive	To store quantity of medicine, tablets and strips
Float	Primitive	Storing prices of medicines
Lists	Collection	To hold attributes of a single medicine
String	Primitive	To store medicine name, and brand name
Dictionary	Collection	To map each medicine information with an id

Table 3: Data Structures Used

These specific data types were used to achieve the goals of the project due to the following reasons:

- **Dictionaries with integer keys** provide O (1) time complexity when it is used for indexing to find relevant medicine information.
- **List**: List was used to store relevant information for a single medicine to be mapped with a key in the dictionary as it follows a strict order. Since lists are mutable objects in Python, they can be easily updated at any attribute ease.
- **Int**: Integer data types were used for quantity of items as quantity cannot be a decimal number.
- **Float** data type was used to store price information of medicine as price can be both integers i.e. whole numbers or floating point numbers.
- **String** was used to store brand name and medicine name.

5. Program

The program follows the paradigm of procedural programming, where blocks of code are divided into functions. The program is designed to try to read the given file path of the program, which is initialized on the top of the program, the file path of the program is a string which Python tries to open using open () function. If the program successfully reads opens the file the contents of the file are returned in a dictionary, else the program returns to an empty dictionary.

The display function of the program takes the function which reads the file `read_file()` as argument i.e. callback function and stores the returned value into a variable.

5.1 Medicine Info File

Medicine info file contains inventory information about MedStore Pvt Ltd where the information is in comma separated format (CSV). The first column contains medicine name, second column contains brand name, third column contains medicine quantity in tablets, rate of tablets, rate of strips, number of tablets per strip. The position of these is arranged in the format specified in the coursework guidelines.

```
Paracetamol 500mg, Lomus, 1200, 5, 45, 10
Cetirizine 10mg, Quest, 800, 4, 35, 12
Amoxicillin 500mg, Nepal Remedies, 500, 12, 110, 15
Pantoprazole 40mg, Deurali-Janta, 600, 7, 60, 8
ORS Sachet, Time Pharma, 300, 20, 180, 12
Azithromycin 500mg, Lomus, 450, 3, 220, 8
Ibuprofen 400mg, Quest, 950, 6, 28, 20
Ciprofloxacin 500mg, Nepal Remedies, 320, 9, 85, 10
Diclofenac 50mg, Deurali-Janta, 700, 4, 40, 15
Omeprazole 20mg, Time Pharma, 600, 5, 55, 12
Losartan 50mg, Lomus, 500, 8, 90, 7
Metformin 500mg, Quest, 1100, 3, 25, 25
Doxycycline 100mg, Nepal Remedies, 400, 7, 110, 9
Amlodipine 5mg, Deurali-Janta, 750, 6, 45, 14
Cetirizine 5mg, Time Pharma, 880, 2, 30, 18
Paracetamol 250mg, Lomus, 1500, 4, 25, 30
Amoxiclav 625mg, Quest, 300, 10, 180, 6
Pantoprazole 20mg, Nepal Remedies, 520, 5, 45, 10
Ofloxacin 200mg, Deurali-Janta, 410, 8, 95, 11
Montelukast 10mg, Time Pharma, 260, 6, 130, 5
Atenolol 50mg, Lomus, 620, 7, 65, 9
Hydrochlorothiazide 25mg, Quest, 900, 4, 35, 16
Cloxacillin 500mg, Nepal Remedies, 340, 11, 105, 7
Rabeprazole 20mg, Deurali-Janta, 580, 5, 70, 13
Fexofenadine 120mg, Time Pharma, 470, 6, 140, 8
Cefixime 200mg, Lomus, 290, 9, 210, 4
Tramadol 50mg, Quest, 510, 3, 75, 10
Moxifloxacin 400mg, Nepal Remedies, 230, 12, 250, 3
Glibenclamide 5mg, Deurali-Janta, 680, 4, 40, 12
Loratadine 10mg, Time Pharma, 820, 5, 28, 22
Salbutamol 2mg, Lomus, 540, 6, 32, 15
Erythromycin 250mg, Quest, 370, 8, 115, 7
Fluconazole 150mg, Nepal Remedies, 280, 7, 160, 5
Naproxen 250mg, Deurali-Janta, 630, 5, 55, 11
Domperidone 10mg, Time Pharma, 720, 4, 38, 14
Cephalexin 500mg, Lomus, 390, 10, 125, 6
Clopidogrel 75mg, Quest, 460, 6, 95, 8
Metronidazole 400mg, Nepal Remedies, 850, 5, 42, 20
Amlodipine 10mg, Deurali-Janta, 590, 7, 60, 9
```

Figure 4: Medicine_info.txt File

5.2 Read File Function

Read file function takes the file path as string as argument, it tries to open the file in read mode if the file opens successfully the information is extracted and stored in lists and dictionary however if the file fails to open it returns an error saying the file couldn't be opened.

```python
path = "medicine_info.txt"

def read_file(path):
    """
    Reads a given file and returns the contents of the file in a dictionary format

    args:
        path : a string which contains the file path of the file which is to be read

    returns:
        med_info : a dictionary with indexes such as 1, 2, 3 as keys and medicine information as values in a list
                example: 1 : [name, brand_name, quantity, rate per strip, rate per tablet, number of tablet per strip]
    """
    try:
        f = open(path, "r")
    except FileNotFoundError as e:
        print(f"Error: {e}")
        return {}
    lines = f.readlines()
    med_info = {} # Stores the med information in the format 1 : [med information]
    idx = 0 # Stores the index for medicine information
    for line in lines:
        idx += 1
        line = line.replace("\n", " ") # Replace new line character and store it in line for parsing later on
        content = line.split(",")
        med_name = content[0]
        brand_name = content[1]
        med_quantity_tablets = int(content[2])
        rate_tablet = float(content[3])
        rate_strip = float(content[4])
        num_tablet_per_strip = int(content[5])
        med_info[idx] = [med_name, brand_name, med_quantity_tablets, rate_tablet, rate_strip, num_tablet_per_strip]
    return med_info
```

Figure 5: Read File Function - Program 1

5.3 Display Function

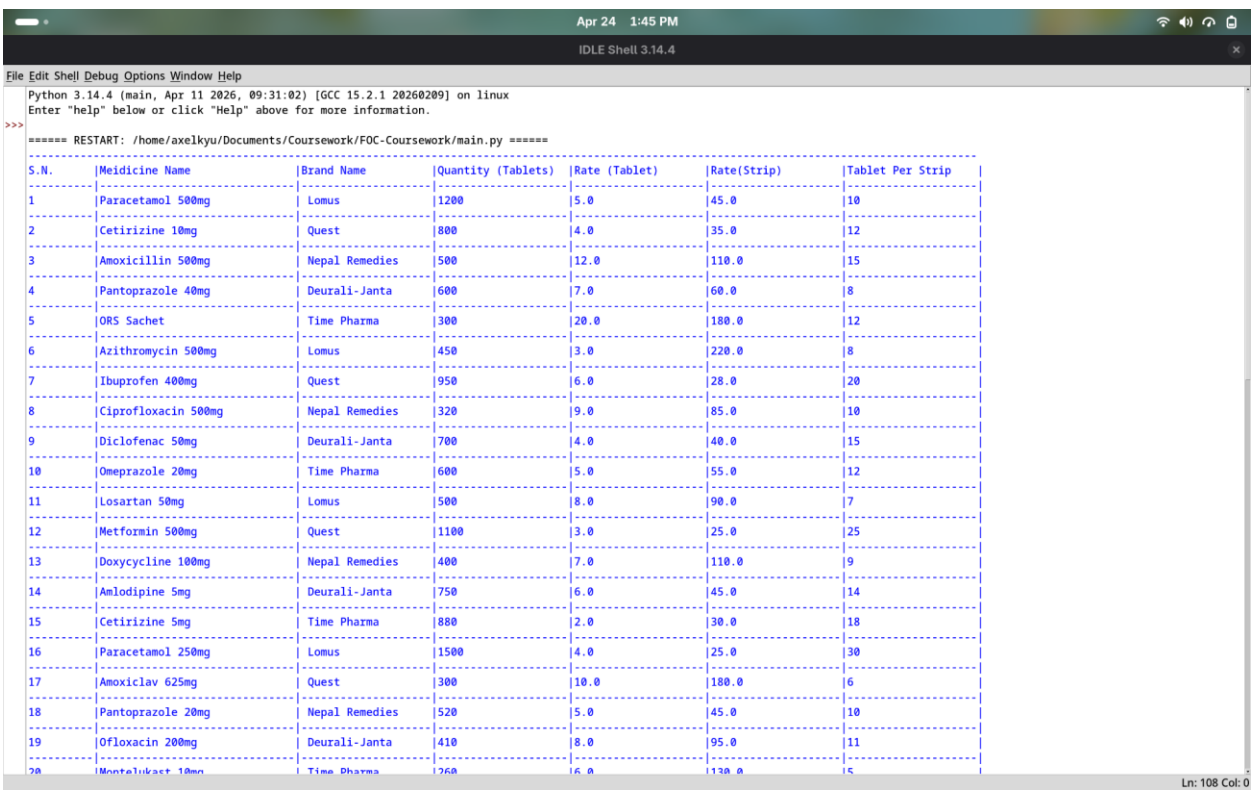
The display function callbacks read file function and stores it in a variable which if is none data type is detected due to file failing to open it is checked and nothing is displayed however if data is not empty, data is shown in a tabular format.

```python
def display(read_file):
    """
    Chains the function from read file to display the contents of the file

    args:
    read_file: Function call back to call an independent function
    inside another independent function

    output:
        Output of medicine information in a tabular format
    """
    display_content = read_file(path)

    if display_content is None:
        print("No Data To Display")
        return

    print("-"*146)
    print(f"{"S.N.":<10}|{"Meidicine Name":30}|{"Brand Name":<20}|{"Quantity (Tablets)":<20}|{"Rate (Tablet)":<20}|{"Rate(Strip)":<20}|{"Tablet Per Strip":<20}|")
    print("-"*10 + "|" + "-" *30 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" * 20 + "|")
    for key, values in display_content.items():
        print(f"{key:<10}|{values[0]:<30}|{values[1]:<20}|{values[2]:<20}|{values[3]:<20}|{values[4]:<20}|{values[5]:<20}|")
        print("-"*10 + "|" + "-" *30 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" *20 + "|" + "-" * 20 + "|")


display(read_file)
```

Figure 6: Display Function - Program 2

5.4 Output

After display function is called in the main the output is displayed in a tabular format where the user can interact with the program.



```
Python 3.14.4 (main, Apr 11 2026, 09:31:02) [GCC 15.2.1 20260209] on linux
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: /home/axelkyu/Documents/Coursework/FOC-Coursework/main.py ======
--------------------------------------------------------------------------------------------------------------------------------
S.N.      |Meidicine Name                |Brand Name          |Quantity (Tablets)  |Rate (Tablet)       |Rate(Strip)         |Tablet Per Strip    |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
1         |Paracetamol 500mg             | Lomus              |1200                |5.0                 |45.0                |10                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
2         |Cetirizine 10mg               | Quest              |800                 |4.0                 |35.0                |12                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
3         |Amoxicillin 500mg             | Nepal Remedies     |500                 |12.0                |110.0               |15                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
4         |Pantoprazole 40mg             | Deurali-Janta      |600                 |7.0                 |60.0                |8                   |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
5         |ORS Sachet                    | Time Pharma        |300                 |20.0                |180.0               |12                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
6         |Azithromycin 500mg            | Lomus              |450                 |3.0                 |220.0               |8                   |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
7         |Ibuprofen 400mg               | Quest              |950                 |6.0                 |28.0                |20                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
8         |Ciprofloxacin 500mg           | Nepal Remedies     |320                 |9.0                 |85.0                |10                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
9         |Diclofenac 50mg               | Deurali-Janta      |700                 |4.0                 |40.0                |15                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
10        |Omeprazole 20mg               | Time Pharma        |600                 |5.0                 |55.0                |12                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
11        |Losartan 50mg                 | Lomus              |500                 |8.0                 |90.0                |7                   |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
12        |Metformin 500mg               | Quest              |1100                |3.0                 |25.0                |25                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
13        |Doxycycline 100mg             | Nepal Remedies     |400                 |7.0                 |110.0               |9                   |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
14        |Amlodipine 5mg                | Deurali-Janta      |750                 |6.0                 |45.0                |14                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
15        |Cetirizine 5mg                | Time Pharma        |880                 |2.0                 |30.0                |18                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
16        |Paracetamol 250mg             | Lomus              |1500                |4.0                 |25.0                |30                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
17        |Amoxiclav 625mg               | Quest              |300                 |10.0                |180.0               |6                   |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
18        |Pantoprazole 20mg             | Nepal Remedies     |520                 |5.0                 |45.0                |10                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
19        |Ofloxacin 200mg               | Deurali-Janta      |410                 |8.0                 |95.0                |11                  |
----------|------------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
20        |Montelukast 10mg              | Time Pharma        |260                 |6.0                 |130.0               |5                   |
```

Figure 7: Output of Program

6. Conclusion

The first milestone of the project was to read the contents of the file, store it in an appropriate Python data structure i.e. and display the contents of the file. This milestone was achieved by developing two functions as shown in section **5.2** and **5.3**.

The milestone was a good learning experience for tinkering with file handling in Python, as a text file was opened in Python using read only mode the contents of the file were retrieved and parsed using appropriate primitive data types such as int, float and strings, after parsing the data type were stored in collection data types such as dictionary and list. This milestone also was a big step for following procedural programming paradigm i.e. using functions to divide the program into different sections. An important concept of call back function, function passed into another function as an argument is defined as function callback, was used to call the read file function into display function to display the contents of the file. Exception handling was also used to check if the file exists or not where if the exists the program executes as normal else the program returns a message saying a file was not found and displays no data.

6.1 Challenges Faced

The challenges faced whilst developing the program till this milestone are listed below:

- **File Not Found Error** when the file name is initialized due to incorrect naming the file was not found and was raising errors which was later fixed using try-catch to gracefully handle file not found exceptions.
- **None Type Error** when file was not found the read file function was raising an error as the expected return type was dictionary however none type data was returned which caused “none type error” which was resolved by returning an empty dictionary instead of None or empty return statement.
- **Display Formatting** to display the data in a tabular format visual error were faced due to incorrect spacing where the program was showing data in uneven order due to improper indexing this was resolved by proper spaces and using a single print statement to display the contents in a tabular format.

7. Bibliography

American National Standard, 1973. *Flowchart Symbols and their usage in information processing.* 1 ed. Washington D.C.: American National Standard.

Floyd, R. W., 2007. The Paradigms of Programming. *Stanford Uniovesity,* 1(1), p. 7.

ISO, 1998. *Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts.* 1 ed. Massachusets : ISO.

Kelly, S., 2019. What is Python?. *Springer Nature,* 26 5, 1(1), pp. 5-9.

Knuth, D. E., 1997. *The Art of Computer Programming, Volume 1: Fundamental Algorithms.* 3 ed. Massachusetts: Addison-Wesley.

Sayeth Sabbith, M. F. T., 2019. Python Current Trend and Application an Overview. *International Journal of Advance Engineering and Research Development,* 6(10), p. 7.